0312. Burst Balloons

Hard · 2-D Dynamic Programming
leetcode.com/problems/burst-balloons

Problem

- Bursting balloon i earns nums[left] nums[i] nums[right] (current neighbours; outside = 1). Maximise total coins.

Intuition

- Choosing the first balloon to burst makes subproblems depend on each other. Instead choose the LAST balloon k to burst in an interval (l, r): its neighbours at that moment are exactly l and r.
- Dp[l][r] = max over k in (l, r) of dp[l][k] + a[l]a[k]a[r] + dp[k][r], with a padded by 1 on both ends.

Brute Force

- Try every burst order: O(n!).

Optimal Approach

- A = [1] + nums + [1]; dp over open intervals.
- For len = 2..n+1, for l, r = l + len: try every k between.
- Answer dp[0][n+1].

Worked Example

```
[3,1,5,8] -> 167.

padded a = [1,3,1,5,8,1]
best order: burst 1, 5, 3, 8
3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 15+120+24+8 = 167
```

Java Solution

```java
public class BurstBalloons {
    public int maxCoins(int[] nums) {
        int n = nums.length;
        int[] a = new int[n + 2];
        a[0] = a[n + 1] = 1;
        System.arraycopy(nums, 0, a, 1, n);
        int[][] dp = new int[n + 2][n + 2];
        for (int len = 2; len <= n + 1; len++)
            for (int l = 0; l + len <= n + 1; l++) {
                int r = l + len;
                for (int k = l + 1; k < r; k++)
                    dp[l][r] = Math.max(dp[l][r], dp[l][k] + a[l] * a[k] * a[r] + dp[k][r]);
            }
        return dp[0][n + 1];
    }
}
```

Complexity

- Time: O(n^3). Space: O(n^2).

Pitfalls

- Iterate by interval length so smaller intervals are ready.